

Chapter 18 — Recursion

0. What You Will Learn in This Chapter

Recursion is one of the most important foundations for trees, graphs, backtracking, dynamic programming, divide and conquer, and linked-list problems.

A recursive function solves a problem by solving smaller versions of the same problem.

By the end of this chapter, you should understand:

- What recursion is
- Why recursion exists
- How recursive calls use the call stack
- How to define a base case
- How to define a recursive case
- How to trace recursion
- How recursion works on linked lists
- How recursion works on trees
- How recursion works on arrays
- How recursion connects to backtracking
- How recursion connects to dynamic programming
- How recursion connects to divide and conquer
- How to avoid infinite recursion
- How to analyze recursive time and space complexity
- When recursion is useful
- When iteration is better

Core problems covered in this chapter:

1. Factorial
 2. Fibonacci Number
 3. Power of Two
 4. Reverse String
 5. Reverse Linked List, recursive
 6. Swap Nodes in Pairs, recursive
 7. Merge Two Sorted Lists, recursive
 8. Maximum Depth of Binary Tree
 9. Invert Binary Tree
 10. Same Tree
 11. Symmetric Tree
 12. Path Sum
 13. Pascal's Triangle II
 14. Pow(x, n)
 15. Climbing Stairs, recursion to DP preview
 16. Generate Parentheses, recursion to backtracking preview
 17. Subsets, recursion to backtracking preview
-

1. What Is Recursion?

Recursion is when a function calls itself.

Example:

```
void countdown(int n) {  
    if (n == 0) {  
        return;  
    }  
  
    cout << n << endl;  
  
    countdown(n - 1);  
}
```

Call:

```
countdown(3);
```

Output:

```
3  
2  
1
```

The function keeps calling itself with a smaller value until it reaches the base case.

2. Why Recursion Exists

Some problems naturally contain smaller versions of themselves.

Example:

To compute factorial:

```
5! = 5 * 4!  
4! = 4 * 3!  
3! = 3 * 2!  
2! = 2 * 1!  
1! = 1
```

So:

```
factorial(n) = n * factorial(n - 1)
```

This is recursive.

Trees are also recursive:

```
A tree = root + left subtree + right subtree
```

So many tree problems are naturally recursive.

3. Core Components of Recursion

Every recursive solution needs two parts.

3.1 Base Case

The stopping condition.

Without a base case, recursion never ends.

Example:

```
if (n == 0) {  
    return 1;  
}
```

3.2 Recursive Case

The part where function calls itself on a smaller or simpler input.

Example:

```
return n * factorial(n - 1);
```

4. Core Need / Why Use Recursion Over Iteration?

Recursion is useful when the problem has nested or branching structure.

4.1 Why Not Iteration?

Iteration is good for linear problems.

Recursion is often cleaner for:

- trees
- divide and conquer
- nested structures
- generating combinations
- DFS
- backtracking
- linked-list recursive rewiring

4.2 Why Not Stack Manually?

Recursion uses the system call stack automatically.

You can convert recursion to iterative stack, but recursive code is often simpler.

Example:

Tree DFS is usually easier recursively.

4.3 Why Not Dynamic Programming Immediately?

Some DP problems start as recursion.

You first define the recursive relation, then optimize with memoization.

Example:

Fibonacci:

```
fib(n) = fib(n - 1) + fib(n - 2)
```

Then memoize.

5. When to Use Recursion

Use recursion when:

5.1 Problem Can Be Split Into Smaller Same Problems

Examples:

- factorial
- Fibonacci

- power
- merge sort
- binary tree depth

5.2 Tree or Graph DFS

Examples:

- Maximum Depth of Binary Tree
- Same Tree
- Path Sum
- Invert Binary Tree

5.3 Backtracking / Generate All Possibilities

Examples:

- Subsets
- Permutations
- Generate Parentheses
- Combination Sum

5.4 Divide and Conquer

Examples:

- Merge Sort
- Quick Sort
- Pow(x, n)

5.5 Linked List Reversal or Pair Operations

Examples:

- Reverse Linked List recursively
 - Swap Nodes in Pairs recursively
 - Merge Two Sorted Lists recursively
-

6. When NOT to Use Recursion

6.1 Recursion Depth Too Large

If input size can be huge, recursion may cause stack overflow.

Example:

Linked list with 100,000 nodes.

Iterative version may be safer.

6.2 Simple Linear Loop Is Clearer

Do not force recursion for simple sum/count traversal.

6.3 Repeated Subproblems Without Memoization

Naive Fibonacci recursion is very slow.

Use memoization or tabulation.

6.4 Need Strict $O(1)$ Call Stack Space

Recursion uses stack space.

If $O(1)$ space is required, use iteration.

7. Mental Models

7.1 Trust the Function

When writing recursion, assume your function already works for smaller input.

Example:

To reverse a list:

Assume reverseList(head->next) reverses the rest.

Then attach head at the end.

7.2 Domino Model

Each recursive call pushes the next domino.

Base case stops the chain.

Then returns unwind backward.

7.3 Call Stack Model

Each recursive call waits for the next call to finish.

Example:

```
factorial(4)
  waits for factorial(3)
    waits for factorial(2)
      waits for factorial(1)
```

Then results return upward.

7.4 Tree Model

Branching recursion forms a recursion tree.

Fibonacci:

```
fib(5)
-> fib(4)
-> fib(3)
```

Each call can branch into more calls.

8. Pattern Recognition Checklist

Ask:

18. Can the problem be expressed using smaller version of itself?
19. Is there a natural base case?
20. Does the input shrink each call?
21. Is the data structure recursive, like a tree?
22. Do I need DFS?
23. Do I need generate all valid possibilities?
24. Is divide and conquer natural?
25. Will recursion depth be safe?
26. Are there repeated subproblems?
27. Should I add memoization?

If yes, recursion may apply.

9. C++ Tools Needed

9.1 Function Calling Itself

```
int solve(int n) {
  if (n == 0) {
```

```
        return 0;
    }

    return solve(n - 1);
}
```

9.2 Passing by Reference

Avoid copying large vectors/strings:

```
void dfs(vector<int>& nums, int index)
```

9.3 Helper Functions

Often LeetCode method calls a private helper:

```
class Solution {
public:
    int mainFunction(...) {
        return helper(...);
    }

    int helper(...) {
        ...
    }
};
```

9.4 Recursion With Return Value

```
int depth(TreeNode* root) {
    if (root == nullptr) {
        return 0;
    }

    return 1 + max(depth(root->left), depth(root->right));
}
```

9.5 Recursion With Side Effects

```
void dfs(TreeNode* root, vector<int>& result) {
    if (root == nullptr) {
        return;
    }

    result.push_back(root->val);
    dfs(root->left, result);
    dfs(root->right, result);
}
```

10. Core Templates

Template 1 — Linear Recursion

```
void solve(int i, int n) {  
    if (i == n) {  
        return;  
    }  
  
    // process i  
  
    solve(i + 1, n);  
}
```

Template 2 — Recursion With Return

```
int solve(int n) {  
    if (n == 0) {  
        return baseValue;  
    }  
  
    return combine(n, solve(n - 1));  
}
```

Template 3 — Binary Tree Recursion

```
int dfs(TreeNode* root) {  
    if (root == nullptr) {  
        return 0;  
    }  
  
    int left = dfs(root->left);  
    int right = dfs(root->right);  
  
    return combine(root, left, right);  
}
```

Template 4 — Tree DFS With Result Vector

```
void dfs(TreeNode* root, vector<int>& result) {  
    if (root == nullptr) {  
        return;  
    }  
}
```

```
result.push_back(root->val);

dfs(root->left, result);
dfs(root->right, result);
}
```

Template 5 — Linked List Recursive Pattern

```
ListNode* solve(ListNode* head) {
    if (head == nullptr || head->next == nullptr) {
        return head;
    }

    ListNode* newHead = solve(head->next);

    // reconnect head

    return newHead;
}
```

Template 6 — Divide and Conquer

```
int solve(vector<int>& nums, int left, int right) {
    if (left == right) {
        return nums[left];
    }

    int mid = left + (right - left) / 2;

    int leftAnswer = solve(nums, left, mid);
    int rightAnswer = solve(nums, mid + 1, right);

    return combine(leftAnswer, rightAnswer);
}
```

Template 7 — Recursion to Backtracking

```
void backtrack(vector<int>& nums, int index, vector<int>& path) {
    if (index == nums.size()) {
        // record path
        return;
    }
}
```



```

// not choose
backtrack(nums, index + 1, path);

// choose
path.push_back(nums[index]);
backtrack(nums, index + 1, path);
path.pop_back();
}

```

11. Major Variations of Recursion

Linear recursion	one recursive call	factorial
Branching recursion	multiple calls	Fibonacci
Tree recursion	left + right subtree	max depth
Linked-list recursion	solve rest then reconnect	reverse list
Divide and conquer	split input	Pow, merge sort
Backtracking recursion	choose/explore/unchoose	subsets
DFS recursion	visit connected structure	tree/graph DFS
Memoized recursion	cache overlapping subproblems	Fibonacci DP

12. Full Problem Walkthrough 1 — Factorial

12.1 Problem Summary

Compute:

$$n! = n * (n - 1) * \dots * 1$$

Example:

$$5! = 120$$

12.2 How to Identify Recursion

Factorial naturally depends on smaller factorial:

$$\text{factorial}(n) = n * \text{factorial}(n - 1)$$

12.3 C++ Code

```
int factorial(int n) {  
    if (n == 0 || n == 1) {  
        return 1;  
    }  
  
    return n * factorial(n - 1);  
}
```

12.4 Dry Run

```
factorial(4)  
= 4 * factorial(3)  
= 4 * 3 * factorial(2)  
= 4 * 3 * 2 * factorial(1)  
= 24
```

12.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

because of call stack.

12.6 Common Mistake

No base case causes infinite recursion.

13. Full Problem Walkthrough 2 — Fibonacci Number

13.1 Problem Summary

Compute nth Fibonacci number:

```
fib(0) = 0  
fib(1) = 1  
fib(n) = fib(n - 1) + fib(n - 2)
```

13.2 How to Identify Recursion

Each Fibonacci number depends on two smaller Fibonacci numbers.

13.3 Naive Recursive Code

```
int fib(int n) {
    if (n <= 1) {
        return n;
    }

    return fib(n - 1) + fib(n - 2);
}
```

13.4 Problem With Naive Recursion

It recomputes the same values many times.

Example:

```
fib(5)
calls fib(4) and fib(3)
fib(4) also calls fib(3)
```

Repeated subproblems cause exponential time.

13.5 Complexity

Naive time:

$O(2^n)$

Space:

$O(n)$

13.6 Memoized Version Preview

```
int helper(int n, vector<int>& memo) {
    if (n <= 1) {
        return n;
    }

    if (memo[n] != -1) {
        return memo[n];
    }

    memo[n] = helper(n - 1, memo) + helper(n - 2, memo);

    return memo[n];
}

int fib(int n) {
    vector<int> memo(n + 1, -1);
```

```
    return helper(n, memo);  
}
```

Time becomes:

$O(n)$

This is the bridge from recursion to dynamic programming.

14. Full Problem Walkthrough 3 — LeetCode 231: Power of Two

14.1 Problem Summary

Return true if integer n is a power of two.

Example:

$n = 16$

Output:

true

14.2 Recursive Intuition

A number is power of two if:

- it is 1
- or it is even and $n / 2$ is power of two

14.3 C++ Code

```
class Solution {  
public:  
    bool isPowerOfTwo(int n) {  
        if (n == 1) {  
            return true;  
        }  
  
        if (n <= 0 || n % 2 != 0) {  
            return false;  
        }  
  
        return isPowerOfTwo(n / 2);  
    }  
};
```

14.4 Complexity

Time:

$O(\log n)$

Space:

$O(\log n)$

14.5 Alternative

Bit manipulation is better:

```
return n > 0 && (n & (n - 1)) == 0;
```

But recursion is useful for learning.

15. Full Problem Walkthrough 4 — LeetCode 344: Reverse String, Recursive

15.1 Problem Summary

Reverse character array in-place.

Example:

```
['h','e','l','l','o']
```

Output:

```
['o','l','l','e','h']
```

15.2 Recursive Intuition

Swap outer characters, then recursively reverse inner substring.

15.3 C++ Code

```
class Solution {
public:
    void helper(vector<char>& s, int left, int right) {
        if (left >= right) {
            return;
        }

        swap(s[left], s[right]);

        helper(s, left + 1, right - 1);
    }

    void reverseString(vector<char>& s) {
        helper(s, 0, s.size() - 1);
    }
};
```

```
}  
};
```

15.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

because recursion depth is $O(n)$.

15.5 Iterative Version

Two pointers are usually better for $O(1)$ space.

16. Full Problem Walkthrough 5 — LeetCode 206: Reverse Linked List, Recursive

16.1 Problem Summary

Reverse linked list.

Input:

1 -> 2 -> 3 -> nullptr

Output:

3 -> 2 -> 1 -> nullptr

16.2 Recursive Intuition

Trust recursion:

```
reverseList(head->next)
```

reverses the rest of the list.

Then attach head after its next node.

16.3 C++ Code

```
class Solution {  
public:  
    ListNode* reverseList(ListNode* head) {  
        if (head == nullptr || head->next == nullptr) {  
            return head;  
        }  
    }  
};
```

```
ListNode* newHead = reverseList(head->next);

head->next->next = head;
head->next = nullptr;

return newHead;
}
};
```

16.4 Dry Run

Input:

1 -> 2 -> 3

Base case returns 3.

Unwind:

For node 2:

2->next->next = 2

2->next = nullptr

List:

3 -> 2

For node 1:

3 -> 2 -> 1

16.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

due to recursion stack.

16.6 Common Mistakes

- Forgetting head->next = nullptr
- Returning head instead of newHead
- Not handling empty list

17. Full Problem Walkthrough 6 — LeetCode 24: Swap Nodes in Pairs, Recursive

17.1 Problem Summary

Swap every two adjacent nodes.

Example:

1 -> 2 -> 3 -> 4

Output:

2 -> 1 -> 4 -> 3

17.2 Recursive Intuition

Swap first two nodes.

Then recursively swap the rest.

17.3 C++ Code

```
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        if (head == nullptr || head->next == nullptr) {
            return head;
        }

        ListNode* second = head->next;
        ListNode* rest = swapPairs(second->next);

        second->next = head;
        head->next = rest;

        return second;
    }
};
```

17.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

17.5 Common Mistakes

- Losing rest of list
 - Returning old head
 - Not handling one remaining node
-

18. Full Problem Walkthrough 7 — LeetCode 21: Merge Two Sorted Lists, Recursive

18.1 Problem Summary

Merge two sorted linked lists.

18.2 Recursive Intuition

If one list is empty, return the other.

Otherwise, choose smaller head and recursively merge the rest.

18.3 C++ Code

```
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        if (list1 == nullptr) {
            return list2;
        }

        if (list2 == nullptr) {
            return list1;
        }

        if (list1->val <= list2->val) {
            list1->next = mergeTwoLists(list1->next, list2);
            return list1;
        } else {
            list2->next = mergeTwoLists(list1, list2->next);
            return list2;
        }
    }
};
```

18.4 Complexity

Time:

$O(n + m)$

Space:

$O(n + m)$

due to recursion stack.

19. Full Problem Walkthrough 8 — LeetCode 104: Maximum Depth of Binary Tree

19.1 Problem Summary

Return maximum depth of binary tree.

Example:

```
  3
 / \
9  20
 / \
15 7
```

Output:

3

19.2 How to Identify Recursion

A tree is recursive.

Depth of tree:

```
1 + max(depth(left), depth(right))
```

19.3 C++ Code

```
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (root == nullptr) {
            return 0;
        }

        int leftDepth = maxDepth(root->left);
        int rightDepth = maxDepth(root->right);

        return 1 + max(leftDepth, rightDepth);
    }
};
```

```
}  
};
```

19.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

where h is tree height.

Worst case:

$O(n)$

Balanced tree:

$O(\log n)$

20. Full Problem Walkthrough 9 — LeetCode 226: Invert Binary Tree

20.1 Problem Summary

Invert a binary tree.

Example:

```
  4  
 / \  
2   7
```

Output:

```
  4  
 / \  
7   2
```

20.2 Recursive Intuition

Invert left subtree.

Invert right subtree.

Swap children.

20.3 C++ Code

```
class Solution {  
public:  
    TreeNode* invertTree(TreeNode* root) {  
        if (root == nullptr) {
```

```
        return nullptr;
    }

    TreeNode* left = invertTree(root->left);
    TreeNode* right = invertTree(root->right);

    root->left = right;
    root->right = left;

    return root;
}
};
```

20.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

21. Full Problem Walkthrough 10 — LeetCode 100: Same Tree

21.1 Problem Summary

Return true if two binary trees are structurally identical and node values are equal.

21.2 Recursive Intuition

Two trees are same if:

- 28. both roots are null
- 29. both roots are non-null and values equal
- 30. left subtrees are same
- 31. right subtrees are same

21.3 C++ Code

```
class Solution {
public:
    bool isSameTree(TreeNode* p, TreeNode* q) {
        if (p == nullptr && q == nullptr) {
            return true;
        }
    }
};
```

```

    if (p == nullptr || q == nullptr) {
        return false;
    }

    if (p->val != q->val) {
        return false;
    }

    return isSameTree(p->left, q->left) &&
           isSameTree(p->right, q->right);
}
};

```

21.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

22. Full Problem Walkthrough 11 — LeetCode 101: Symmetric Tree

22.1 Problem Summary

Return true if binary tree is mirror of itself.

22.2 Recursive Intuition

Left and right subtrees are mirrors if:

- left value equals right value
- left-left mirrors right-right
- left-right mirrors right-left

22.3 C++ Code

```

class Solution {
public:
    bool isMirror(TreeNode* a, TreeNode* b) {
        if (a == nullptr && b == nullptr) {
            return true;
        }

        if (a == nullptr || b == nullptr) {

```

```

        return false;
    }

    if (a->val != b->val) {
        return false;
    }

    return isMirror(a->left, b->right) &&
        isMirror(a->right, b->left);
}

bool isSymmetric(TreeNode* root) {
    if (root == nullptr) {
        return true;
    }

    return isMirror(root->left, root->right);
}
};

```

22.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

23. Full Problem Walkthrough 12 — LeetCode 112: Path Sum

23.1 Problem Summary

Return true if tree has root-to-leaf path with sum equal to target.

23.2 Recursive Intuition

At each node, subtract node value from target.

At leaf, check if remaining target equals leaf value.

23.3 C++ Code

```

class Solution {
public:
    bool hasPathSum(TreeNode* root, int targetSum) {

```

```
    if (root == nullptr) {  
        return false;  
    }  
  
    if (root->left == nullptr && root->right == nullptr) {  
        return targetSum == root->val;  
    }  
  
    int remaining = targetSum - root->val;  
  
    return hasPathSum(root->left, remaining) ||  
        hasPathSum(root->right, remaining);  
}  
};
```

23.4 Complexity

Time:

$O(n)$

Space:

$O(h)$

24. Full Problem Walkthrough 13 — LeetCode 50: Pow(x, n)

24.1 Problem Summary

Implement:

pow(x, n)

Example:

x = 2
n = 10

Output:

1024

24.2 How to Identify Recursion

Power can be solved by divide and conquer.

$x^n = (x^{(n/2)})^2$

If n is odd:

$$x^n = x * (x^{(n/2)})^2$$

24.3 C++ Code

```
class Solution {
public:
    double fastPow(double x, long long n) {
        if (n == 0) {
            return 1.0;
        }

        double half = fastPow(x, n / 2);

        if (n % 2 == 0) {
            return half * half;
        }

        return half * half * x;
    }

    double myPow(double x, int n) {
        long long power = n;

        if (power < 0) {
            x = 1 / x;
            power = -power;
        }

        return fastPow(x, power);
    }
};
```

24.4 Why Use long long?

n can be INT_MIN.

Negating INT_MIN overflows int.

So convert to long long.

24.5 Complexity

Time:

$O(\log n)$

Space:

$O(\log n)$

25. Full Problem Walkthrough 14 — LeetCode 70: Climbing Stairs

25.1 Problem Summary

You can climb 1 or 2 steps.

Return number of distinct ways to reach top.

Example:

$n = 3$

Ways:

1+1+1

1+2

2+1

Output:

3

25.2 Recursive Intuition

To reach step n , previous step could be:

- $n - 1$
- $n - 2$

So:

$\text{ways}(n) = \text{ways}(n - 1) + \text{ways}(n - 2)$

25.3 Naive Recursion

```
int climbStairs(int n) {  
    if (n <= 2) {  
        return n;  
    }  
  
    return climbStairs(n - 1) + climbStairs(n - 2);  
}
```

This is exponential.

25.4 Memoized Recursion

```
class Solution {  
public:  
    int helper(int n, vector<int>& memo) {
```

```

    if (n <= 2) {
        return n;
    }

    if (memo[n] != -1) {
        return memo[n];
    }

    memo[n] = helper(n - 1, memo) + helper(n - 2, memo);

    return memo[n];
}

int climbStairs(int n) {
    vector<int> memo(n + 1, -1);
    return helper(n, memo);
}
};

```

25.5 Complexity

With memoization:

Time:

$O(n)$

Space:

$O(n)$

25.6 Recursion to DP

This problem belongs mainly in DP, but recursion helps derive the recurrence.

26. Full Problem Walkthrough 15 — LeetCode 118: Pascal's Triangle

26.1 Problem Summary

Generate first numRows of Pascal's Triangle.

Example:

numRows = 5

Output:

[
[1],

```
[1,1],  
[1,2,1],  
[1,3,3,1],  
[1,4,6,4,1]  
]
```

26.2 Recursive Intuition

Each row depends on previous row.

26.3 C++ Code

```
class Solution {  
public:  
    vector<vector<int>> generate(int numRows) {  
        if (numRows == 1) {  
            return {{1}};  
        }  
  
        vector<vector<int>> result = generate(numRows - 1);  
  
        vector<int> prev = result.back();  
        vector<int> current(numRows, 1);  
  
        for (int i = 1; i < numRows - 1; i++) {  
            current[i] = prev[i - 1] + prev[i];  
        }  
  
        result.push_back(current);  
  
        return result;  
    }  
};
```

26.4 Complexity

Time:

$O(\text{numRows}^2)$

Space:

$O(\text{numRows}^2)$

27. Full Problem Walkthrough 16 — Generate Parentheses Preview

27.1 Problem Summary

Generate all valid parentheses combinations of n pairs.

Example:

$n = 3$

Output:

["((()))","(()())","(())()","()(())","()()()"]

27.2 Recursive Intuition

At every step, choose:

- add (if open count < n
- add) if close count < open count

This is recursion with choices, also called backtracking.

27.3 C++ Code

```
class Solution {
public:
    void dfs(int n, int open, int close, string& path, vector<string>& result) {
        if (path.size() == 2 * n) {
            result.push_back(path);
            return;
        }

        if (open < n) {
            path.push_back('(');
            dfs(n, open + 1, close, path, result);
            path.pop_back();
        }

        if (close < open) {
            path.push_back(')');
            dfs(n, open, close + 1, path, result);
            path.pop_back();
        }
    }

    vector<string> generateParenthesis(int n) {
        vector<string> result;
```

```
    string path;

    dfs(n, 0, 0, path, result);

    return result;
}
};
```

27.4 Why This Is Preview

This belongs mainly to Backtracking.
But backtracking is built on recursion.

28. Full Problem Walkthrough 17 — Subsets Preview

28.1 Problem Summary

Given unique integers, return all subsets.
Example:

```
nums = [1,2]
```

Output:

```
[[], [1], [2], [1,2]]
```

28.2 Recursive Intuition

For each element, choose:

- exclude it
 - include it
-

28.3 C++ Code

```
class Solution {
public:
    void dfs(vector<int>& nums, int index, vector<int>& path,
vector<vector<int>>& result) {
        if (index == nums.size()) {
            result.push_back(path);
            return;
        }

        dfs(nums, index + 1, path, result);

        path.push_back(nums[index]);
```

```

        dfs(nums, index + 1, path, result);
        path.pop_back();
    }

    vector<vector<int>> subsets(vector<int>& nums) {
        vector<vector<int>> result;
        vector<int> path;

        dfs(nums, 0, path, result);

        return result;
    }
};

```

28.4 Why This Is Preview

Subsets is a backtracking problem.

This chapter focuses on recursion fundamentals.

29. Mini Problem 1 — Sum of Array Recursively

```

int sumArray(vector<int>& nums, int index) {
    if (index == nums.size()) {
        return 0;
    }

    return nums[index] + sumArray(nums, index + 1);
}

```

30. Mini Problem 2 — Print Linked List Recursively

```

void printList(ListNode* head) {
    if (head == nullptr) {
        return;
    }

    cout << head->val << endl;
}

```

```
    printList(head->next);  
}
```

31. Mini Problem 3 — Print Linked List in Reverse

```
void printReverse(ListNode* head) {  
    if (head == nullptr) {  
        return;  
    }  
  
    printReverse(head->next);  
    cout << head->val << endl;  
}
```

Key idea:
Process after recursive call.

32. Mini Problem 4 — Sum Root-to-Leaf DFS Concept

```
void dfs(TreeNode* root, int currentSum) {  
    if (root == nullptr) {  
        return;  
    }  
  
    currentSum += root->val;  
  
    if (root->left == nullptr && root->right == nullptr) {  
        // use currentSum  
        return;  
    }  
  
    dfs(root->left, currentSum);  
    dfs(root->right, currentSum);  
}
```

33. Common Edge Cases for Recursion

33.1 Missing Base Case

Causes infinite recursion.

33.2 Base Case Too Late

If you dereference null before checking null, code crashes.

Wrong:

```
if (root->left == nullptr)
```

before checking:

```
root == nullptr
```

33.3 Empty Input

Handle:

- empty list
- empty tree
- empty array
- empty string

33.4 One Element

Often base case.

33.5 Negative Exponent

For Pow(x, n), handle negative n.

33.6 Stack Overflow

Very deep recursion can exceed call stack.

34. Common Mistakes and Wrong Code

Mistake 1 — No Progress Toward Base Case

Wrong:

```
return solve(n);
```

Correct:

```
return solve(n - 1);
```

Mistake 2 — Forgetting Return Value

Wrong:

```
solve(root->left);  
solve(root->right);
```

when you need combine results.

Correct:


```
return 1 + max(solve(root->left), solve(root->right));
```

Mistake 3 — Shared State Not Restored

In backtracking preview, after push:

```
path.push_back(x);  
dfs(...);  
path.pop_back();
```

Need pop.

Mistake 4 — Exponential Recursion Without Memoization

Naive Fibonacci and Climbing Stairs are exponential.

Add memoization.

Mistake 5 — Returning Wrong Head in Recursive Linked List

For reverse linked list, return newHead, not old head.

35. Interview Explanation Templates

35.1 General Recursion

I define a base case for the smallest input. Then I assume the recursive function correctly solves smaller inputs and use those results to solve the current input.

35.2 Tree Recursion

For each node, I recursively solve the left subtree and right subtree. Then I combine those answers with the current node's value.

35.3 Linked List Recursion

I recursively solve the rest of the list and then reconnect the current node based on the problem requirement.

35.4 Divide and Conquer

I split the problem into smaller parts, solve each recursively, and combine the results.

35.5 Recursion to DP

If the recursive function calls the same state multiple times, I store results in a memo table to avoid recomputation.

36. Complexity Patterns

Linear recursion	$O(n)$	$O(n)$
Binary tree DFS	$O(n)$	$O(h)$

Recursion Type	Time	Space
Balanced tree DFS	$O(n)$	$O(\log n)$ stack
Skewed tree DFS	$O(n)$	$O(n)$ stack
Naive Fibonacci	$O(2^n)$	$O(n)$
Memoized Fibonacci	$O(n)$	$O(n)$
Fast power	$O(\log n)$	$O(\log n)$
Generate subsets	$O(2^n * n)$	$O(n)$ path stack
Generate parentheses	Catalan count	$O(n)$ path stack
Merge sort recursion	$O(n \log n)$	$O(\log n)$ recursion stack

37. Time-Boxed Practice Guidance

Easy Recursion

Target:

10–20 minutes

Examples:

- Factorial
- Fibonacci
- Power of Two
- Reverse String
- Maximum Depth of Binary Tree

Medium Recursion

Target:

20–40 minutes

Examples:

- Reverse Linked List recursive
- Swap Nodes in Pairs
- Same Tree
- Symmetric Tree
- Path Sum
- Pow(x, n)

Harder Recursion / Recursion-to-Backtracking

Target:

40–60 minutes

Examples:

- Generate Parentheses
- Subsets
- Recursive tree path problems

- Basic recursion-to-DP problems
-

38. Expanded Practice Set

38.1 Must-Do Easy

- 32. Fibonacci Number
- 33. Power of Two
- 34. Reverse String
- 35. Maximum Depth of Binary Tree
- 36. Same Tree
- 37. Symmetric Tree
- 38. Path Sum
- 39. Invert Binary Tree

38.2 Must-Do Medium

- 40. Reverse Linked List, recursive
- 41. Swap Nodes in Pairs
- 42. Merge Two Sorted Lists, recursive
- 43. Pow(x, n)
- 44. Climbing Stairs, memoized recursion
- 45. Pascal's Triangle
- 46. Validate Binary Search Tree, recursive preview
- 47. Binary Tree Paths

38.3 Strong Interview Practice

- 48. Generate Parentheses
- 49. Subsets
- 50. Permutations
- 51. Combination Sum
- 52. Word Search
- 53. Diameter of Binary Tree
- 54. Lowest Common Ancestor
- 55. Path Sum II

38.4 Hard / Stretch

- 56. N-Queens
 - 57. Sudoku Solver
 - 58. Expression Add Operators
 - 59. Basic Calculator, recursive parsing
 - 60. Different Ways to Add Parentheses
 - 61. Burst Balloons, recursion to interval DP
-

39. Suggested Study Order

Step 1 — Basic Linear Recursion

Start with:

- 62. Factorial
- 63. Fibonacci
- 64. Reverse String
- 65. Power of Two

Goal:

Understand base case and recursive case.

Step 2 — Linked List Recursion

Solve:

- 66. Reverse Linked List recursive
- 67. Swap Nodes in Pairs
- 68. Merge Two Sorted Lists recursive

Goal:

Understand recursive pointer rewiring.

Step 3 — Tree Recursion

Solve:

- 69. Maximum Depth
- 70. Invert Binary Tree
- 71. Same Tree
- 72. Symmetric Tree
- 73. Path Sum

Goal:

Learn left/right subtree recursion.

Step 4 — Divide and Conquer

Solve:

- 74. Pow(x, n)
- 75. Pascal's Triangle
- 76. Merge Sort concept

Goal:

Understand split and combine.

Step 5 — Recursion to Backtracking / DP

Solve:

- 77. Generate Parentheses
- 78. Subsets
- 79. Climbing Stairs with memoization

Goal:

See how recursion becomes backtracking and DP.

40. Pattern Decision Table

n depends on n-1	linear recursion	factorial
n depends on n-1 and n-2	branching recursion	Fibonacci
left and right subtree	tree recursion	max depth
reverse rest of list	linked list recursion	reverse list

Problem Clue	Recursion Type	Example
split into halves	divide and conquer	Pow, merge sort
choose / not choose	backtracking recursion	subsets
repeated states	memoized recursion	climbing stairs
root-to-leaf path	DFS recursion	path sum
nested expression	recursive parsing	calculators

41. Final Cheat Sheet

Base Case + Recursive Case

```
int solve(int n) {
    if (n == base) {
        return baseValue;
    }

    return combine(solve(smallerInput));
}
```

Tree Recursion

```
if (root == nullptr) {
    return baseValue;
}

auto left = dfs(root->left);
auto right = dfs(root->right);

return combine(root, left, right);
```

Linked List Recursion

```
if (head == nullptr || head->next == nullptr) {
    return head;
}

ListNode* newHead = solve(head->next);

// reconnect

return newHead;
```

Backtracking Recursion

```
path.push_back(choice);
dfs(nextState);
path.pop_back();
```

Memoized Recursion

```
if (memo[state] exists) {
    return memo[state];
}

memo[state] = solve(smaller states);

return memo[state];
```

42. What to Master Before Chapter 19

Before moving to Backtracking, make sure you can:

- Define base case
- Define recursive case
- Trace call stack
- Explain recursion in your own words
- Solve simple linear recursion
- Solve tree recursion
- Solve linked-list recursion
- Understand recursive return values
- Know when recursion causes stack overflow
- Recognize repeated subproblems
- Add memoization to recursion
- Understand choose/explore/unchoose idea
- Solve Generate Parentheses preview
- Solve Subsets preview

43. Core Takeaway

Recursion solves a problem by trusting the function to solve smaller versions of the same problem.

Every recursive solution needs:

```
base case
recursive case
progress toward base case
```

Recursion is the foundation for:

- tree DFS
- graph DFS
- backtracking

- dynamic programming
- divide and conquer
- recursive parsing
- linked-list pointer rewiring

The key question is:

Can I define the answer in terms of the same problem on smaller input?

If yes, recursion is likely the right starting point.
